

Lecture 4: Anomaly Detection

Alexis BOGROFF

Course on Unsupervised Learning Problems @Albert School

March 31, 2026

- (1) Context: Outlier vs. Novelty Detection
- (2) One-Class SVM: Bounding the Normal
- (3) Isolation Forests: "Few and Different"
- (4) Local Outlier Factor (LOF): Density-Based Anomalies
- (5) Summary & Business Practice

The Business Context

- In many real-world scenarios, anomalies represent critical, actionable events.
- **Finance:** Credit card fraud detection, unusual trading patterns.
- **Industry:** Predictive maintenance (detecting sensor anomalies before a machine breaks).
- **Cybersecurity:** Network intrusion detection.
- **Challenge:** Anomalies are extremely rare (often $< 1\%$ of the data), making standard supervised classification fail.

Before diving into specific models, we must understand the core geometric or statistical assumptions they rely on:

- **1. Statistical / Parametric:** Assumes data follows a known distribution (e.g., Gaussian). Anomalies are points in the low-probability tails.
 - *Examples:* Z-Score, Mahalanobis Distance.
- **2. Proximity & Density-Based:** Assumes normal points occur in dense neighborhoods, while anomalies are far or in sparse regions.
 - *Examples:* DBSCAN, k-NN, Local Outlier Factor (LOF).
- **3. Boundary / Domain-Based:** Learns a geometric boundary enclosing the normal data. Anything outside is an anomaly.
 - *Examples:* One-Class SVM.
- **4. Isolation / Ensembles:** Does not model normal data at all. Directly isolates anomalies via random partitioning.
 - *Examples:* Isolation Forest.

Two distinct paradigms in Machine Learning:

1. Outlier Detection (Unsupervised)

- **The Data:** The training dataset contains both normal data and anomalies (it is "polluted").
- **The Goal:** Fit the model on the central mode(s) of the data and isolate the observations that are far away.
- **Algorithms:** Isolation Forest, Local Outlier Factor (LOF).

2. Novelty Detection (Semi-Supervised)

- **The Data:** The training dataset is strictly clean (only normal data).
- **The Goal:** Learn the exact boundary of the "normal" state. When new data arrives in production, flag anything outside this boundary as a "novelty".
- **Algorithms:** One-Class SVM.

Why classic Mahalanobis distance is not enough

If we try to use standard Mahalanobis distance on a dataset that already contains outliers, we run into a critical mathematical flaw:

- **The Masking Effect:** Outliers have a massive gravitational pull on standard statistical metrics. They heavily skew the empirical mean μ and artificially inflate the covariance matrix Σ .
- By stretching the covariance matrix towards themselves, the calculated distance of these outliers to the center drops significantly.
- **Result:** The outliers manipulate the boundary to include themselves. They "mask" their own anomalous nature, leading to False Negatives.

Minimum Covariance Determinant (MCD)

To fix the masking effect, we must compute the Mahalanobis distance without letting outliers influence μ and Σ . In Scikit-Learn, this is implemented as `EllipticEnvelope`.

- **The Goal:** Find a pure, unpolluted subset of h observations (typically $h \approx \frac{n}{2}$) that perfectly represents the dense "Normal" core.
- **The Mathematical Engine:** The algorithm searches for the subset of size h whose covariance matrix has the **smallest possible determinant**.
- *Why the determinant?* The determinant of a covariance matrix measures the geometric volume of the ellipsoid containing the data. Minimizing it forces the algorithm to find the most tightly packed cluster of points, naturally leaving the scattered outliers out of the calculation.

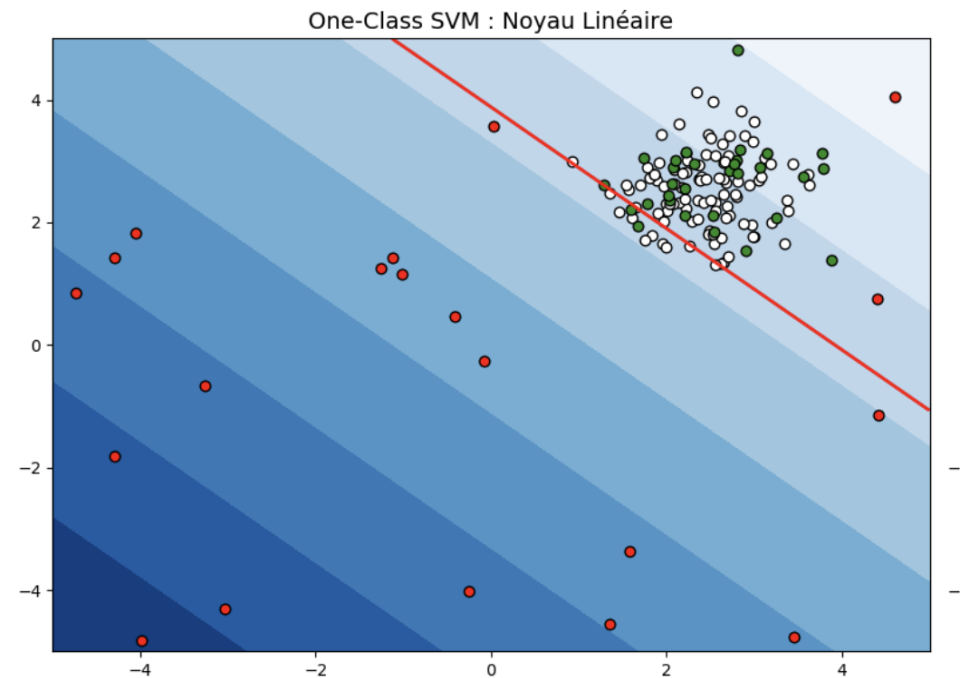
You actually already know how to detect anomalies

- **From Math (Linear Algebra IV):** The Mahalanobis distance $d_M(x_i)$. By calculating the distance of a point to the distribution's center, scaling by the covariance matrix, you can flag points exceeding a χ^2 threshold as outliers.
- **From ML III (Session 2):** DBSCAN. Remember that points failing to be Core or Border points are classified as Noise (Outliers). DBSCAN natively detects anomalies

So why learn new models? Because classical distance-based approaches (e.g. Mahalanobis distance) rely on strong distributional assumptions (e.g. Gaussianity) and degrade in high-dimensional spaces, while density-based methods such as DBSCAN also suffer from the curse of dimensionality.

- (1) Context: Outlier vs. Novelty Detection
- (2) One-Class SVM: Bounding the Normal
- (3) Isolation Forests: "Few and Different"
- (4) Local Outlier Factor (LOF): Density-Based Anomalies
- (5) Summary & Business Practice

- **The Goal:** Instead of separating two distinct classes, **One-Class SVM** learns to draw a tight envelope around a single "Normal" class.
- **The Mechanism:** Data is projected into a high-dimensional space. The algorithm then pushes a hyperplane *as far away from the origin as possible*, stopping only when it bumps into the normal data (Support Vectors).
- **The Result:** Back in our original space, this creates a closed boundary tightly hugging the dense regions. Any point falling outside is flagged as an anomaly.



The Primal Formulation (Schölkopf, 1999)

The algorithm solves an optimization problem to find a hyperplane separated from the origin by a maximum margin ρ :

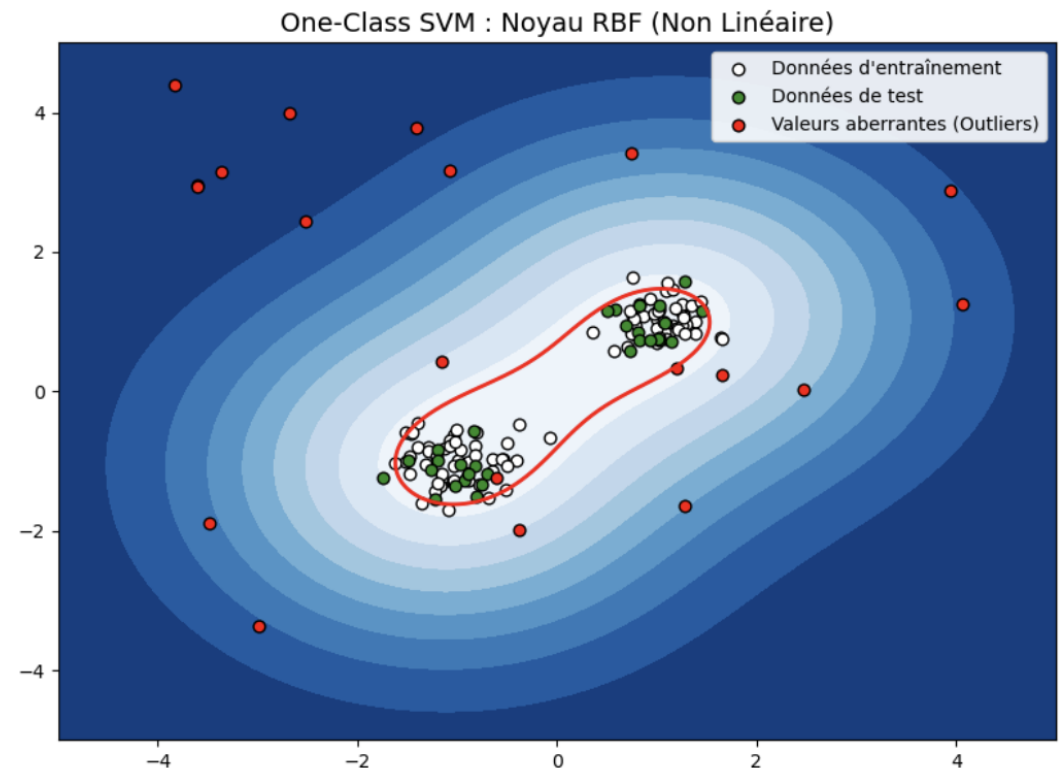
$$\min_{w, \rho, \xi} \frac{1}{2} \|w\|^2 - \rho + \frac{1}{\nu n} \sum_{i=1}^n \xi_i$$

Breaking down the objective:

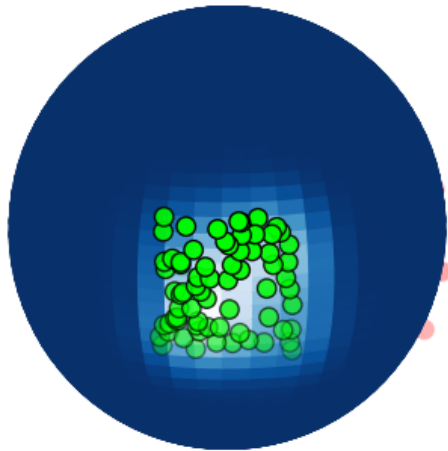
- $\frac{1}{2} \|w\|^2$: The regularization term to keep the margin wide (similar to standard SVM).
- $-\rho$: We want to push the hyperplane as far from the origin as possible (maximize ρ , so minimize $-\rho$).
- ξ_i : **Slack variables**. These allow some points to fall on the wrong side of the boundary (handling training anomalies).
- ν : The trade-off parameter. It controls the maximum fraction of outliers and the minimum fraction of support vectors.

Handling Non-Linear boundaries

- Just like standard SVM, a linear hyperplane is rarely enough.
- We use the **RBF (Radial Basis Function) Kernel** to project data into infinite-dimensional space.
- This allows One-Class SVM to draw tight, complex, non-linear contours around clusters of normal data, no matter their shape.



1. Overfitting (w énorme)



2. Régularisé (w faible)

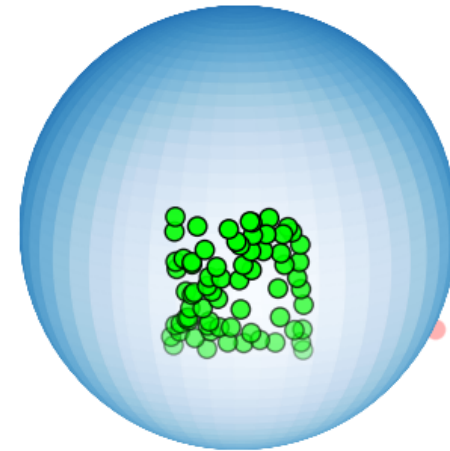


Figure 1: L'effet de la minimisation de w et du paramètre ν sur la fonction de décision.

The Business Reality: How you use it changes everything

1. Novelty Detection (The Ideal Business Case)

- **Setup:** You train *only* on clean, verified "normal" data.
- **Business Example:** Predictive maintenance. You record vibrations of a perfectly healthy jet engine for 100 hours. The OCSVM learns this exact healthy state. Any future deviation is flagged.
- **Advantage:** Highly precise boundary; excellent at detecting subtle, never-before-seen anomalies (Zero-day anomalies).

2. Outlier Detection (The Messy Reality)

- **Setup:** You train on a polluted dataset containing unknown anomalies.
- **Business Example:** Auditing past accounting transactions for fraud without prior labels.
- **Disadvantage:** OCSVM is highly sensitive to outliers during training. If anomalies are in the training set, the boundary might expand to include them, leading to missed frauds (False Negatives).

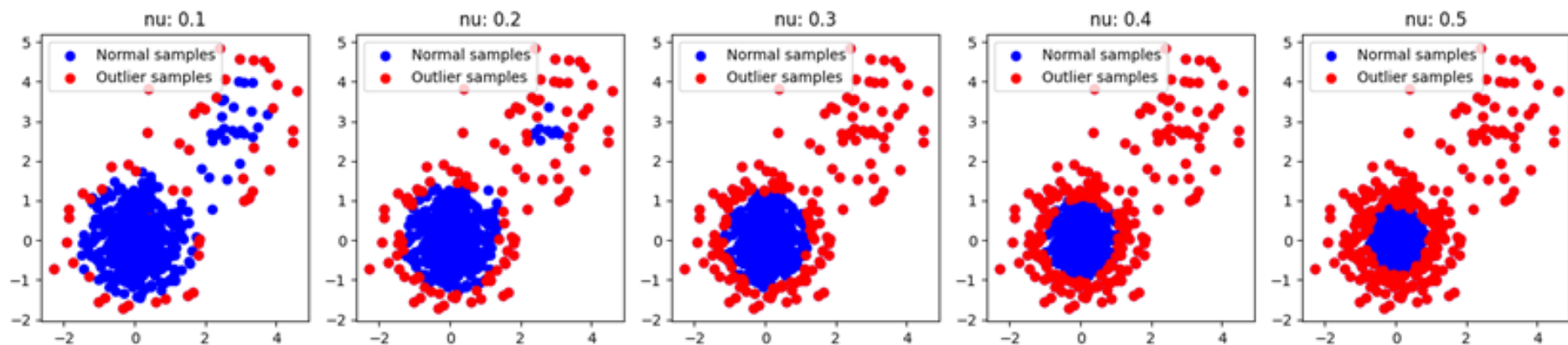
Controlling the Business Trade-off

- The hyperparameter ν (**Nu**) represents the *margin of tolerance* for outliers in your training data.
- **Low ν (e.g., 0.01)**: Strict boundary. You trust your training data is 99% clean.
- **High ν (e.g., 0.10)**: Looser boundary. You admit up to 10% of your training data might be anomalies.

Business Implication

Tuning ν is a direct trade-off between **Investigation Cost** and **Risk**. A high ν means your operations team will receive many alerts (High False Positives, high cost of manual review). A low ν means fewer alerts, but you risk missing a critical failure.

- **Low ν (Wider Boundary):** The contour expands to encompass almost all training data. It is highly inclusive but risks capturing actual anomalies (False Negatives).
- **High ν (Tighter Boundary):** The contour shrinks around the densest core of the data, actively leaving out the specified fraction ν . It is highly exclusive and risks flagging normal variations (False Positives).
- Using the RBF kernel, any non-normally distributed data points will make **the boundary highly irregular**. Tuning ν is what prevents the RBF contour from dangerously over-fitting to noisy, scattered points at the edges of your clusters.



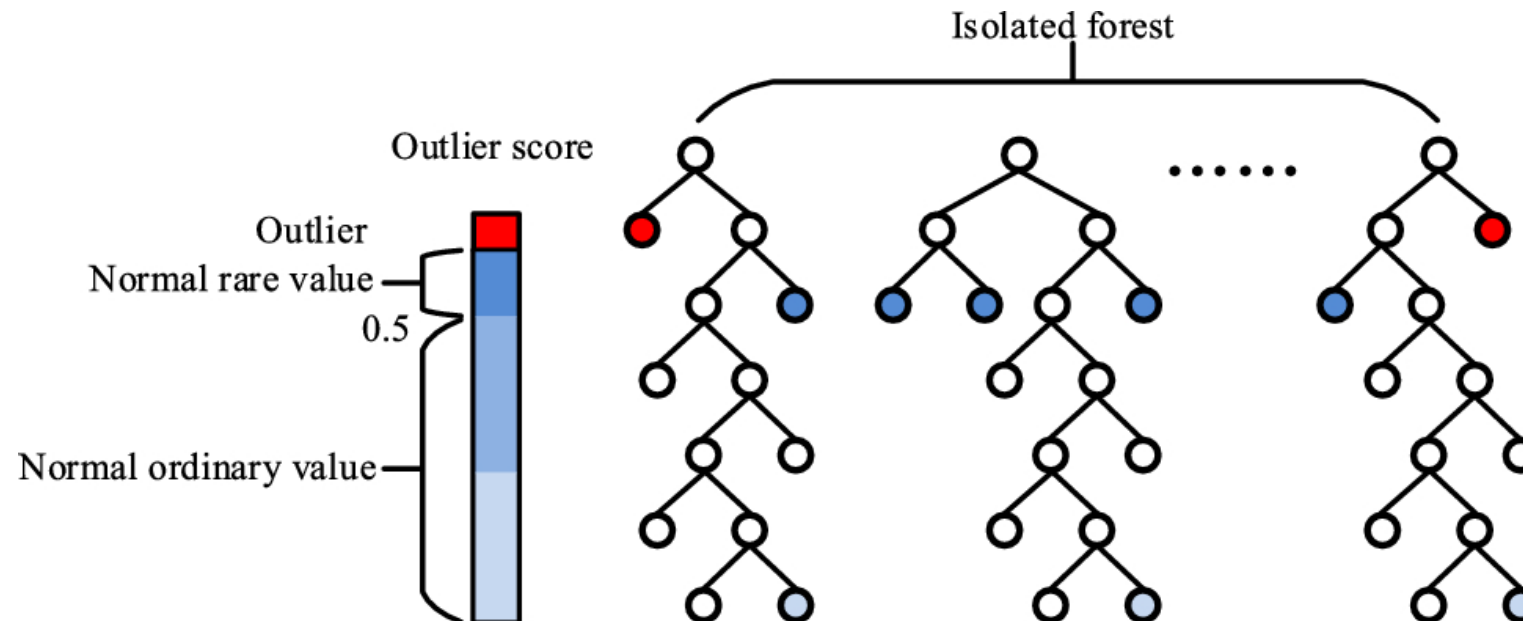
- (1) Context: Outlier vs. Novelty Detection
- (2) One-Class SVM: Bounding the Normal
- (3) Isolation Forests: "Few and Different"
- (4) Local Outlier Factor (LOF): Density-Based Anomalies
- (5) Summary & Business Practice

A paradigm shift in Anomaly Detection

- Most algorithms try to profile the "Normal" data, and then define anomalies as points that don't fit the profile.
- **Isolation Forest (iForest)** does the exact opposite: It explicitly isolates anomalies rather than profiling normal points.
- **Core Intuition:** Anomalies are "*Few and Different*". Because they are rare and distinct, they are much easier to separate from the rest of the dataset.

How do we isolate points? With Random Trees

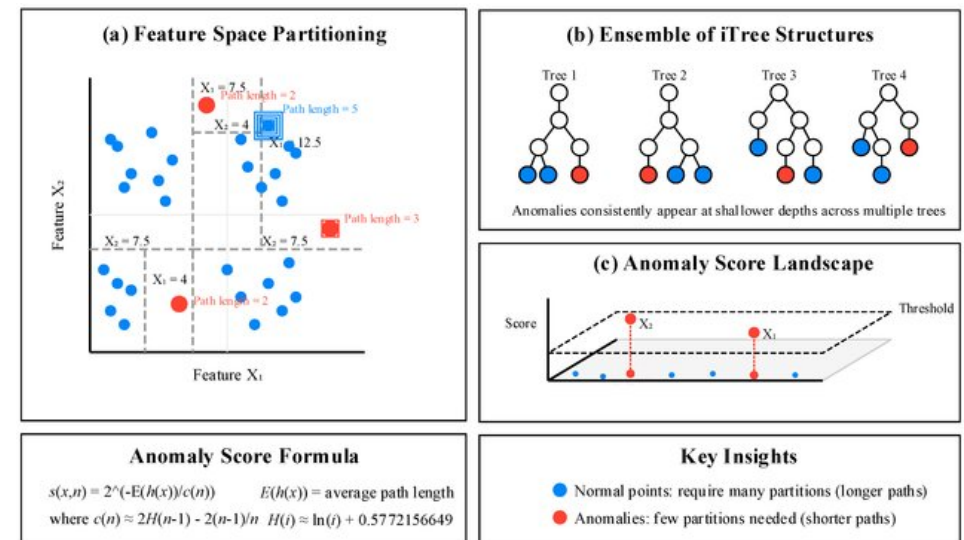
1. Randomly select a feature (e.g., "Transaction Amount").
2. Randomly select a split value between the minimum and maximum values of that feature.
3. This splits the dataset in two.
4. Repeat this process recursively until every single data point is isolated in its own leaf node.



Visualizing the Random Splits

Think of the random tree splits as orthogonal cuts in a 2D feature space:

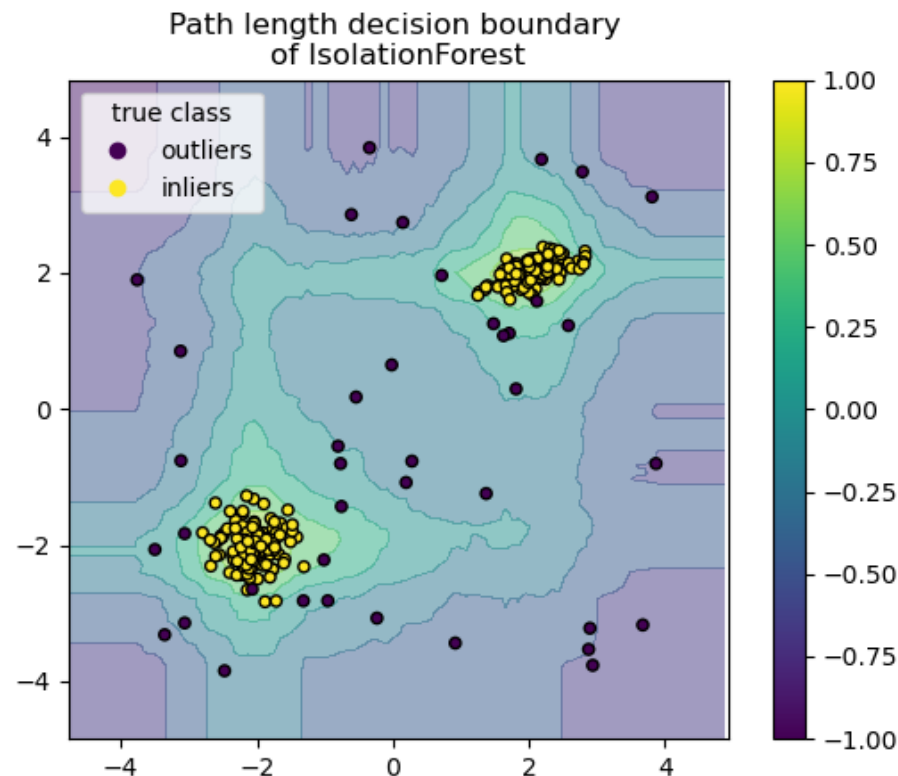
- **Normal Points (Dense Region):**
Deeply buried in the center. It requires many random cuts to finally isolate one specific normal point from its neighbors.
- **Anomalies (Sparse Region):** Far out on the edges. Because the space around them is empty, just 1 or 2 random cuts are often enough to isolate them in their own region.



Conclusion: The number of splits = The path length in the tree.

The Key Insight: Path Length as an Anomaly Score

- The average path length $E(h(x))$ across all trees becomes our primary metric.
- **Anomalies:** Short path lengths (isolated quickly).
- **Normal Points:** Long path lengths (require many splits to isolate).



From Path Length to a Standardized Score

Because path lengths vary based on dataset size n , we normalize the expected path length $E(h(x))$ against $c(n)$, the average path length of an unsuccessful search in a Binary Search Tree.

The Anomaly Score $s(x, n)$ for a data point x is:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

Breaking down the variables:

- $E(h(x))$: Average path length for point x across all trees.
- $c(n) = 2H(n-1) - \frac{2(n-1)}{n}$: The normalization factor (where H is the harmonic number).

Interpreting the Score:

- If $s \approx 1$: **Anomaly** (Path length is much shorter than average).
- If $s < 0.5$: **Normal** (Path length is longer than or equal to average).

Bridging the gap from Scores to Binary Labels

- The model natively outputs a continuous Anomaly Score for every point.
- However, in production, we usually need a strict **Normal (1)** vs. **Anomaly (-1)** label.
- **Definition:** The `contamination` hyperparameter is the expected proportion of outliers in your dataset (typically 0.0 to 0.5).
- It dictates exactly where to draw the threshold line to make this conversion.

How does contamination enforce the cutoff?

1. **Calculate:** The model computes anomaly scores for all N data points.
2. **Rank:** It sorts these points from highest anomaly score to lowest.
3. **Slice:** If `contamination` is set to c (e.g., 0.05), it strictly labels the top $c \times N$ points (the top 5%) as anomalies.
4. **Label:** The remainder are labeled as normal.

Choosing the wrong threshold directly impacts performance.

Setting it Too High (e.g., 0.15 when reality is 0.01):

- The model is forced to over-flag.
- **Result:** High *False Positive* rate (normal points flagged as anomalies).

Setting it Too Low (e.g., 0.01 when reality is 0.10):

- The model is restricted to only the most extreme outliers.
- **Result:** High *False Negative* rate (real anomalies go undetected).

How to choose the right value:

- **Domain Knowledge:** The gold standard. Use historical data (e.g., "Historically, 2% of transactions are fraud" → `contamination=0.02`).
- **Scikit-Learn's 'auto':** Uses the theoretical default threshold of 0.5 from the original research paper. Best when you lack prior knowledge.
- **Iterative Tuning:** Start with a conservative estimate (e.g., 0.01), manually review the flagged points, and incrementally increase if you need more sensitivity.

The Industry Standard for Big Data

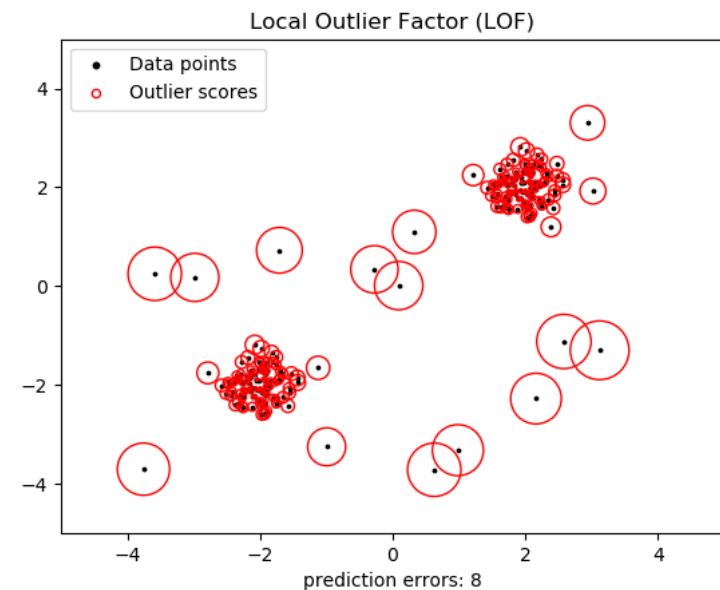
- **Pros:**
 - Extremely fast and highly scalable ($O(n \log n)$).
 - Handles high-dimensional data beautifully.
 - Doesn't rely on distance calculations, making it robust to different scales.
- **Cons:**
 - Can struggle with localized anomalies (anomalies hiding close to dense, normal clusters).

- (1) Context: Outlier vs. Novelty Detection
- (2) One-Class SVM: Bounding the Normal
- (3) Isolation Forests: "Few and Different"
- (4) Local Outlier Factor (LOF): Density-Based Anomalies
- (5) Summary & Business Practice

1. The Initial Distance (Euclidean)

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

- **Theoretically:** A naive approach calculates the distance for every possible pair, leading to a poor scalability of $O(n^2)$.
- **Practically (Scikit-Learn):** It uses smart spatial indexing (**KD-Trees** or **Ball Trees**) to isolate neighborhoods and find nearest neighbors efficiently, bypassing the need to calculate every single distance.



Defining the Neighborhood

We choose a parameter k (the number of neighbors to observe, e.g., $k = 3$).

Definition

The k -**distance** of a point A is the distance separating it from its k -th nearest neighbor.

- **Explanation:** Imagine drawing a circle centered on A that expands until it encompasses exactly k neighbors. The radius of this circle is the k -distance.
- **Role in LOF:** It defines the "neighborhood" of A , denoted $N_k(A)$. All points located inside or exactly on the edge of this circle are considered the k nearest neighbors.

Smoothing the Distances

$$\text{reach-dist}_k(A, B) = \max(k\text{-distance}(B), d(A, B))$$

Why is this "max" function necessary?

- **The "Speed Limit" (Near-Zero):** If A and B are extremely close (a micro-cluster), using their raw distance would cause the calculated density to artificially explode. Replacing it with B 's k -distance limits the maximum possible density.
- **The Edge Case (Perfectly Superimposed):** If k or more points are perfectly stacked, B 's k -distance collapses to exactly 0. The formula outputs $\max(0, 0) = 0$, completely failing to protect the math!
- **The Practical Fix:** To survive true duplicates, libraries like Scikit-Learn do not rely purely on this formula. They add a microscopic ϵ (e.g., 10^{-10}) to the denominator in the next step to prevent division by zero.

Calculating Local Density

Now that we have smoothed distances, we compute the density around our point A .

$$lrd_k(A) = \frac{1}{\frac{\sum_{B \in N_k(A)} \text{reach-dist}_k(A, B)}{|N_k(A)|}}$$

- **Explanation:** The denominator calculates the average reachability distance between A and all its neighbors B . We then take the inverse (1 divided by this average).
- **Role in LOF:** If the average distance to your neighbors is large, the area is sparse. Taking the inverse yields a small density score. Conversely, very close neighbors result in a small average distance, and therefore a high LRD (high density).

The Final Verdict

This final formula compares the density of point A with that of its neighbors to flag anomalies.

$$LOF_k(A) = \frac{\sum_{B \in N_k(A)} \frac{lrd_k(B)}{lrd_k(A)}}{|N_k(A)|}$$

- **Explanation:** For each neighbor B , we calculate the ratio between its density $lrd_k(B)$ and our point's density $lrd_k(A)$. We then average all these ratios.
- **Role in LOF:**
 - $LOF \approx 1$: The density of A is similar to its neighbors (**Normal point**).
 - $LOF \gg 1$: The density of the neighbors is much higher than the density of A . Point A is on the outskirts of a dense group (**Anomaly / Outlier**).

- (1) Context: Outlier vs. Novelty Detection
- (2) One-Class SVM: Bounding the Normal
- (3) Isolation Forests: "Few and Different"
- (4) Local Outlier Factor (LOF): Density-Based Anomalies
- (5) Summary & Business Practice

Python Example

```
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM

# 1. Initialize models with trade-off parameter
iforest = IsolationForest(contamination=0.05, random_state=4)
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
ocsvm = OneClassSVM(nu=0.05, kernel='rbf')

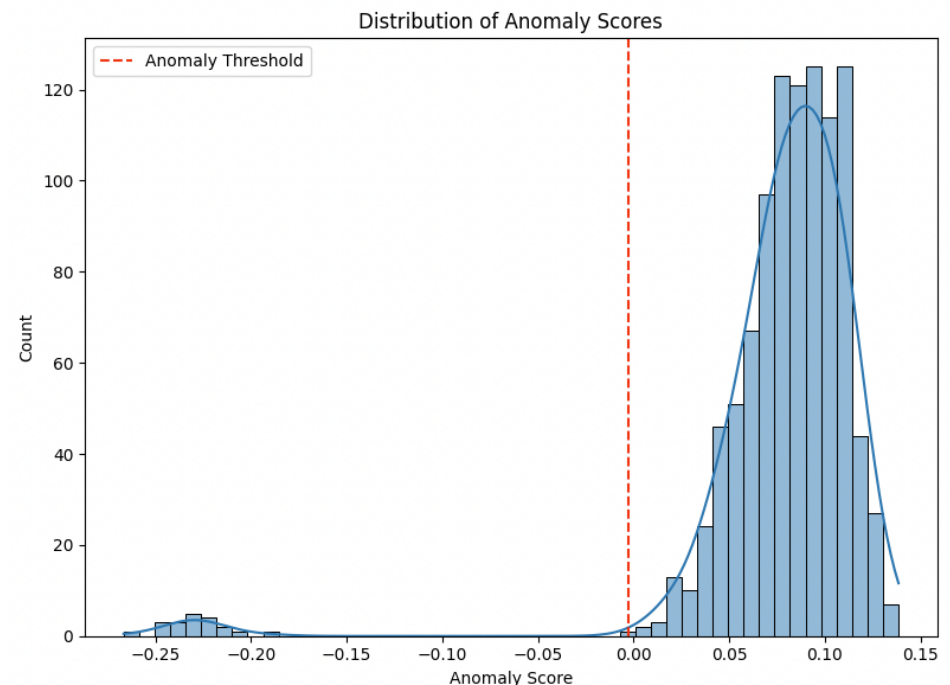
# 2. Fit and Predict
# IF and OCSVM are trained, then used on new data
iforest.fit(X_train)
y_pred = iforest.predict(X_test)

# LOF is unsupervised and predicts on its training set
y_pred_lof = lof.fit_predict(X_train)
```

How contamination works in practice

Before predicting -1 or 1 , models compute a raw continuous Anomaly Score.

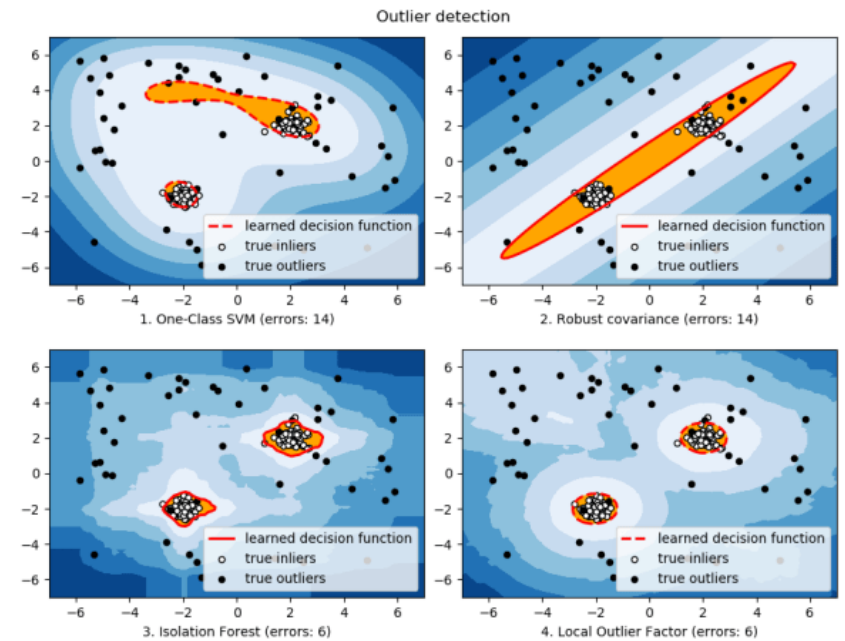
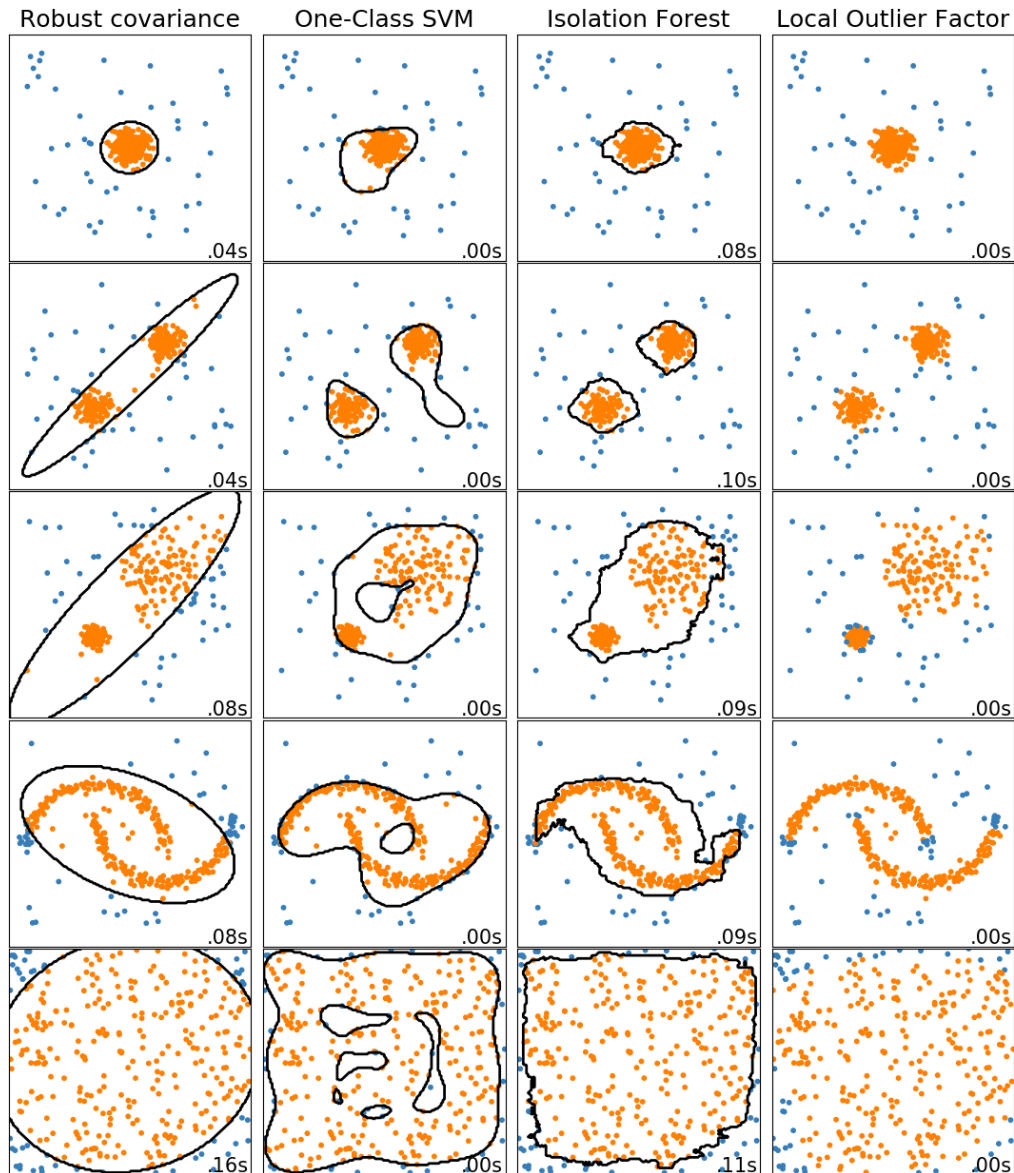
- Plotting the histogram of these scores reveals the dataset's underlying structure.
- **Normal points:** Form the massive central peak(s) (low anomaly score).
- **Anomalies:** Fall into the long, sparse tail on the right.
- The `contamination` parameter simply draws a strict vertical cut-off line across this tail. Everything to the right becomes -1 .



As a Data Scientist, choosing the right model is a business decision:

Model	Core Mechanism	Best Business Fit	Scalability
Isolation Forest	Isolates via random splits	Massive logs, Bot detection, Web traffic	Excellent ($O(n \log n)$)
One-Class SVM	Bounding envelope (RBF)	Predictive Maintenance (Clean training data)	Poor ($O(n^2)$)
LOF	k -NN Density comparison	Contextual fraud, Geospatial anomalies	Moderate to Poor

Empirical Comparison on a Toy Dataset



No one-size-fits-all solution

- **One-Class SVM:** Draws a geometric envelope. It bounds the normal data but can overfit to noise at the edges.
- **Isolation Forest:** Excellent at catching "Global" outliers, but its vertical/horizontal cuts can misclassify points in diagonal sparse spaces.
- **LOF:** The only model that reliably catches "Local" outliers hiding near dense clusters.

Evaluating Unsupervised Models in Business

Because we often lack labels, standard metrics (Accuracy) are useless. We must speak in terms of Cost:

- **The Cost of a False Positive (FP):** The model flags a normal event as an anomaly.
 - *Impact:* Wasted time for investigation teams, "Alert Fatigue", blocking a legitimate customer's credit card (Churn risk).
- **The Cost of a False Negative (FN):** The model misses a true anomaly.
 - *Impact:* A machine breaks down halting production (\$100k/hour), a massive fraud goes undetected, a cyber breach occurs.

Takeaway: Always tune your models (like `contamination` or ν) based on which cost is most dangerous to your specific company.

What we've learned today:

- **The Paradigms:** Outlier Detection (polluted training data) vs. Novelty Detection (strictly clean training data).
- **One-Class SVM:** Learns a non-linear geometric boundary (RBF) around normal data. Perfect for novelty detection, but hyperparameter ν is critical to manage noise.
- **Isolation Forest:** Shifts the mindset by explicitly isolating anomalies using random splits. Highly scalable ($O(n \log n)$) and robust for global outliers.
- **Local Outlier Factor (LOF):** A density-based approach utilizing Reachability Distance. The ultimate tool for finding contextual, local anomalies.
- **The Business Reality:** Unsupervised metrics (like accuracy) are irrelevant. You must tune thresholds (`contamination`) based on the asymmetric financial cost of False Positives vs. False Negatives.

Objective: Apply Anomaly Detection to a highly imbalanced dataset.

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Instructions & Business Questions:

1. Load the dataset and observe the imbalance ratio (Normal vs Fraud).
2. Standardize the features (Required for LOF and EllipticEnvelope, but why not for iForest?).
3. Train an **Isolation Forest** and evaluate its ability to catch fraud without generating too much "Alert Fatigue".
4. Train a **Local Outlier Factor (LOF)** model.
5. Train a **Robust Covariance** model (EllipticEnvelope).
6. **Business Decision:** Compare the three models. Which one scales better in terms of training time? If each False Positive costs 5€ in manual review, and each False Negative costs 500€ in refunded fraud, which model saves the company more money?